

# Relatório prático Microsserviços e Docker

**Repositório:** [github.com/lfbpaiva/DevOps](https://github.com/lfbpaiva/DevOps)

**Branch:** feat/aula04

**Data:** 28 de agosto de 2026

## Objetivo

Implementar a busca de livros por identificador na micro-livraria e executar o serviço de frete em um container Docker. Registrar os comandos utilizados, sua finalidade, os resultados observados e as respectivas evidências.

## Parte 1

Contrato gRPC atualizado, método de busca implementado, rota HTTP disponível e cinco testes de integração aprovados.

## Parte 2

Imagem construída, Shipping executado em container, consulta de frete confirmada pela API e pelo navegador, testes repetidos e limpeza concluída.

## Organização da entrega

O código de aserg-ufmg/micro-livraria foi incorporado ao repositório da disciplina. Essa organização substitui a criação de um fork separado prevista na preparação do roteiro. A licença MIT e os créditos de origem foram mantidos. A main e a branch da Aula 03 não foram alteradas.

## Registro das evidências

As imagens da interface são capturas do navegador. As imagens dos comandos são capturas de um painel local que exibe a saída registrada do PowerShell, e não fotografias da janela do terminal. Saídas longas aparecem em trechos identificados. Os resultados apresentados foram obtidos na execução, não são exemplos de saída esperada.

## Escopo

Foram seguidas as duas tarefas da micro-livraria indicadas no enunciado. O PDF Aula04.pdf foi usado como material de apoio sobre Docker; o exercício de FastAPI, PostgreSQL e Redis descrito nele não integra este relatório.

# 01. Ambiente e repositório

## Comando ou ação executada

```
git remote -v  
git branch --show-current  
node --version  
npm.cmd --version
```

**Explicação:** remote -v mostra os endereços de leitura e envio do repositório. branch --show-current identifica a branch ativa. Os dois últimos comandos verificam as versões do Node.js e do gerenciador de dependências.

**Resultado e interpretação:** O remoto é lfbpaiva/DevOps e a atividade está isolada em feat/aula04. Foram usados Node.js 22.17.0 e npm 10.9.2 no Windows. A main não foi modificada. A opção local safe.directory foi usada quando necessária para acessar esta pasta com o Git.

## Print do resultado

### Registro de execução

Micro-livraria · Aula 04

```
git -c safe.directory='C:/Users/luizf/OneDrive/Área de Trabalho/DevOps' remote -v; git -c  
safe.directory='C:/Users/luizf/OneDrive/Área de Trabalho/DevOps' branch --show-current; node --version;  
npm.cmd --version
```

```
origin https://github.com/lfbpaiva/DevOps.git (fetch)  
origin https://github.com/lfbpaiva/DevOps.git (push)  
feat/aula04  
v22.17.0  
10.9.2
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

## 02. Instalação das dependências

### Comando ou ação executada

```
npm.cmd install
```

**Explicação:** Instala os pacotes declarados em package.json. No PowerShell, npm.cmd chama diretamente o executável de comandos do npm. O arquivo package-lock.json registra as versões resolvidas.

**Resultado e interpretação:** Foram adicionados 299 pacotes. A instalação terminou, mas o npm apontou 30 vulnerabilidades: 5 baixas, 7 moderadas, 15 altas e 3 críticas. O lockfile antigo foi atualizado pelo npm. Não foi aplicado audit fix --force para evitar mudanças incompatíveis com o exercício.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
npm.cmd install
```

```
npm warn old lockfile
```

```
npm warn old lockfile The package-lock.json file was created with an old version of npm,  
npm warn old lockfile so supplemental metadata must be fetched from the registry.
```

```
npm warn old lockfile
```

```
npm warn old lockfile This is a one-time fix-up, please be patient...
```

```
npm warn old lockfile
```

```
added 299 packages, and audited 300 packages in 20s
```

```
36 packages are looking for funding
```

```
run `npm fund` for details
```

```
30 vulnerabilities (5 low, 7 moderate, 15 high, 3 critical)
```

```
To address issues that do not require attention, run:
```

```
npm audit fix
```

```
To address all issues (including breaking changes), run:
```

Saída capturada do comando executado em PowerShell. Trecho 1/2.

## 03. Parte 1 - iniciar os serviços

### Comando ou ação executada

```
npm.cmd run start
```

**Explicação:** Executa o script start de package.json. Nesta primeira etapa, o script inicia frontend, controller, inventory e shipping como processos locais. O nodemon acompanha alterações nos arquivos dos serviços.

**Resultado e interpretação:** Os logs confirmam o frontend em 5000, a API HTTP em 3000, o estoque em 3002 e o frete em 3001. O aviso de verificação de atualização não impediu o início dos serviços. A captura apresenta o trecho dos logs com as portas em funcionamento.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
npm.cmd run start
```

```
Inventory Service running at http://127.0.0.1:3002
Shipping Service running at http://127.0.0.1:3001
Controller Service running on http://127.0.0.1:3000
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
```

Saída capturada do comando executado em PowerShell. Trecho 3/3.

## 04. Parte 1 - contrato da operação

### Comando ou ação executada

```
Get-Content proto/inventory.proto
```

**Explicação:** Exibe o contrato gRPC após a edição. Foi incluída a operação SearchProductByID(Payload), com retorno ProductResponse. A nova mensagem Payload contém um campo id do tipo int32.

**Resultado e interpretação:** O arquivo passou a declarar a operação de busca individual, além da listagem existente. ProductResponse já descrevia os dados de um livro. O contrato define os dados trocados, mas a implementação da busca fica no serviço Inventory.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
Get-Content proto/inventory.proto
```

```
syntax = "proto3";
```

```
service InventoryService {  
    rpc SearchAllProducts(Empty) returns (ProductsResponse) {}  
    rpc SearchProductByID(Payload) returns (ProductResponse) {}  
}
```

```
message Payload {  
    int32 id = 1;  
}
```

```
message Empty{}
```

```
message ProductResponse {  
    int32 id = 1;  
    string name = 2;  
    int32 quantity = 3;  
    float price = 4;
```

Saída capturada do comando executado em PowerShell. Trecho 1/2.

## 05. Parte 1 - implementação da busca

### Comando ou ação executada

```
Select-String -Path services/inventory/index.js, services/controller/index.js -Pattern SearchProductByID -Context 0,9
```

**Explicação:** Localiza o novo método e mostra as linhas seguintes. No Inventory, find procura o produto cujo id corresponde a payload.request.id. No Controller, a rota GET /product/:id chama o método remoto.

**Resultado e interpretação:** O Inventory devolve o livro encontrado pelo callback. O Controller converte o retorno em JSON; falhas de comunicação retornam HTTP 500. A busca foi implementada no backend, como solicitado, sem acrescentar uma interface de pesquisa no frontend.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
Select-String -Path services/inventory/index.js, services/controller/index.js -Pattern SearchProductByID -Context 0,9
```

```
> services\inventory\index.js:22: SearchProductByID: (payload, callback) => {
  services\inventory\index.js:23:     callback(null, products.find((product) => product.id ==
payload.request.id));
  services\inventory\index.js:24: },
  services\inventory\index.js:25:});
  services\inventory\index.js:26:
  services\inventory\index.js:27: server.bindAsync('127.0.0.1:3002', grpc.ServerCredentials.createInsecure(), ()
=> {
  services\inventory\index.js:28:     console.log('Inventory Service running at http://127.0.0.1:3002');
  services\inventory\index.js:29:     server.start();
  services\inventory\index.js:30:});
> services\controller\index.js:40: inventory.SearchProductByID({ id: req.params.id }, (err, product) => {
  services\controller\index.js:41:     if (err) {
  services\controller\index.js:42:         console.error(err);
  services\controller\index.js:43:         res.status(500).send({ error: 'something failed :( ' });
  services\controller\index.js:44:     } else {
  services\controller\index.js:45:         res.json(product);
  services\controller\index.js:46:     }
  services\controller\index.js:47: });
  services\controller\index.js:48:});
```

Saída capturada do comando executado em PowerShell. Trecho 1/2.

#### Registro de execução

Micro-livraria - Aula 04

```
Select-String -Path services/inventory/index.js, services/controller/index.js -Pattern SearchProductByID -Context 0,9
```

```
services\controller\index.js:49:
```

Saída capturada do comando executado em PowerShell. Trecho 2/2.

## 06. Parte 1 - listar os livros

### Comando ou ação executada

```
curl.exe -sS -i http://localhost:3000/products
```

**Explicação:** Envia uma requisição HTTP GET. A opção -i inclui os cabeçalhos na saída; -sS remove o indicador de progresso, mas mantém mensagens de erro. O uso de curl.exe evita ambiguidades com comandos do PowerShell.

**Resultado e interpretação:** A API respondeu HTTP 200 e retornou três produtos, com IDs 1, 2 e 3. O caminho Controller > Inventory está funcionando. Os preços aparecem com pequenas diferenças decimais porque o contrato original usa float.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
curl.exe -sS -i http://localhost:3000/products
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 437
ETag: W/"1b5-H/SdsXWVDtoSiWQ7y1hB40Z8ZQk"
Date: Fri, 28 Aug 2026 19:42:18 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

```
[{"id":1,"name":"Refactoring","quantity":10,"price":79.91999816894531,"photo":"/img/refactoring.png","author":"M
artin Fowler"}, {"id":2,"name":"Engenharia De Software
Moderna","quantity":10,"price":65.9000015258789,"photo":"/img/esm.png","author":"Marco Tulio Valente"},
{"id":3,"name":"Design
Patterns","quantity":10,"price":67.98999786376953,"photo":"/img/design.png","author":"Erich Gamma, John
Vlissides, Richard Helm, Ralph Johnson"}]
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.



## 07. Parte 1 - consultar o produto 1

### Comando ou ação executada

```
curl.exe -sS -i http://localhost:3000/product/1
```

**Explicação:** Acessa o novo endpoint passando o identificador 1 na URL. O Controller encaminha o identificador à operação SearchProductById do Inventory.

**Resultado e interpretação:** A resposta foi HTTP 200 com um objeto, não uma lista: id 1, nome Refactoring e autor Martin Fowler. Isso comprova a busca individual exigida na primeira tarefa. Os testes complementares também verificaram os IDs 2 e 3.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
curl.exe -sS -i http://localhost:3000/product/1
```

HTTP/1.1 200 OK

X-Powered-By: Express

Access-Control-Allow-Origin: \*

Content-Type: application/json; charset=utf-8

Content-Length: 125

ETag: W/"7d-zk09ouY3ZEomyW8hFOD+NwkY"

Date: Fri, 28 Aug 2026 19:42:20 GMT

Connection: keep-alive

Keep-Alive: timeout=5

```
{"id":1,"name":"Refactoring","quantity":10,"price":79.91999816894531,"photo":"/img/refactoring.png","author":"Martin Fowler"}
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.



## 08. Parte 1 - interface da livraria

### Comando ou ação executada

Abrir `http://localhost:5000` no navegador

**Explicação:** O frontend consulta `/products` e cria os cartões dos livros. Esta etapa verifica o sistema pela interface, além dos testes realizados diretamente na API.

**Resultado e interpretação:** Os três livros foram exibidos com suas imagens, autores, preços e controles de frete e compra. O estoque mostrado na interface é um texto fixo do exemplo original; não representa uma atualização dinâmica de estoque.

### Print do resultado

### Micro Livraria

Uma simples livraria utilizando microservices



R\$79.92

Disponível em estoque: 5

**Refactoring**  
Martin Fowler



R\$65.90

Disponível em estoque: 5

**Engenharia De Software Moderna**  
Marco Tulio Valente



R\$67.99

Disponível em estoque: 5

**Design Patterns**  
Erich Gamma, John Vlissides,  
Richard Helm, Ralph Johnson

Feito com ❤ na UFMG

## 09A. Parte 1 - consultar o frete

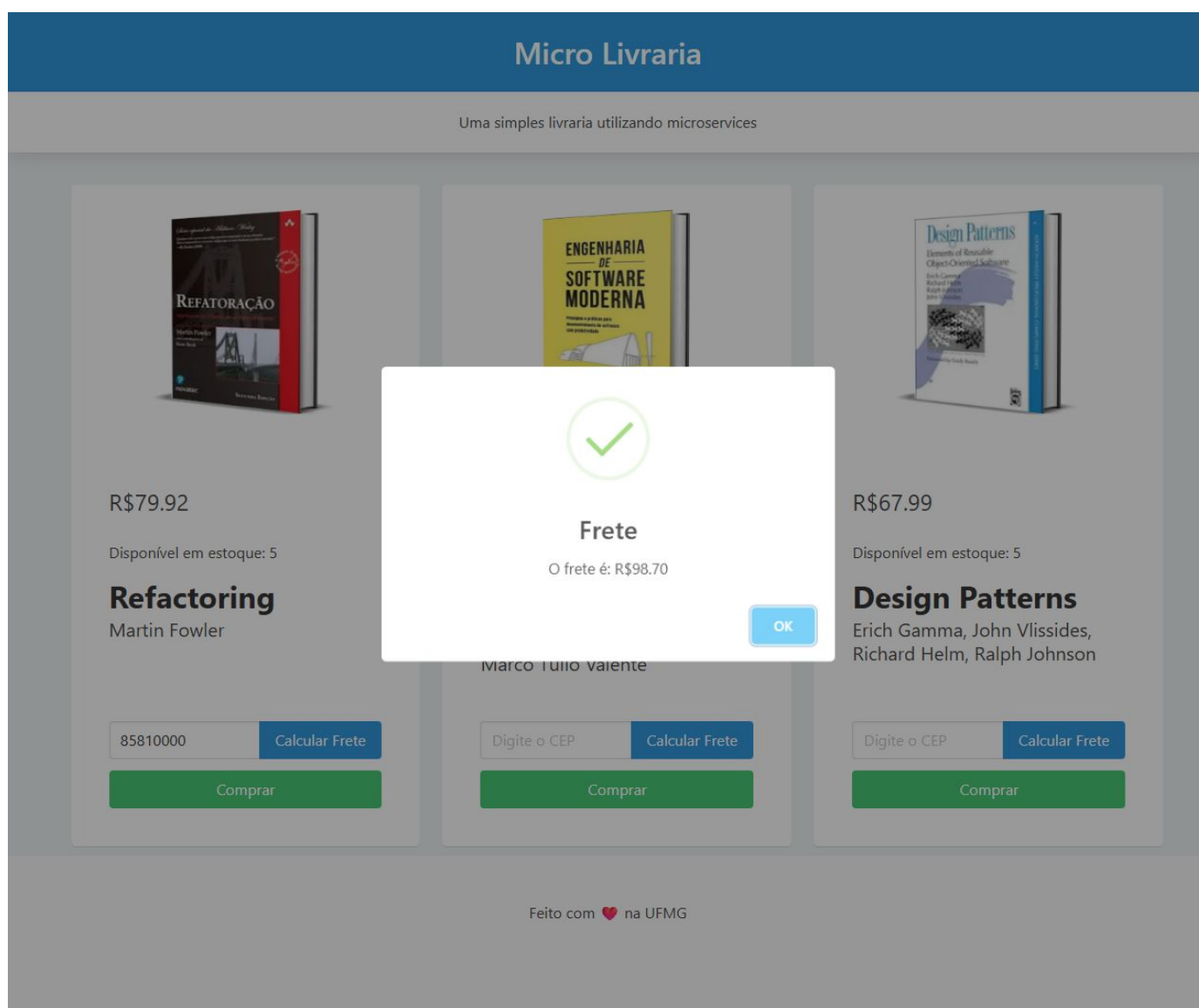
### Comando ou ação executada

Na interface: informar CEP 85810000 > Calcular Frete

**Explicação:** O botão de frete faz uma requisição a /shipping/85810000. A resposta recebida pelo navegador é apresentada em uma mensagem, com o valor formatado em reais.

**Resultado e interpretação:** Com todos os serviços locais, o frete exibido foi R\$ 98,70. O serviço retorna um valor aleatório; não consulta uma transportadora e não calcula o preço a partir do CEP.

### Print do resultado



## 09B. Parte 1 - compra simulada

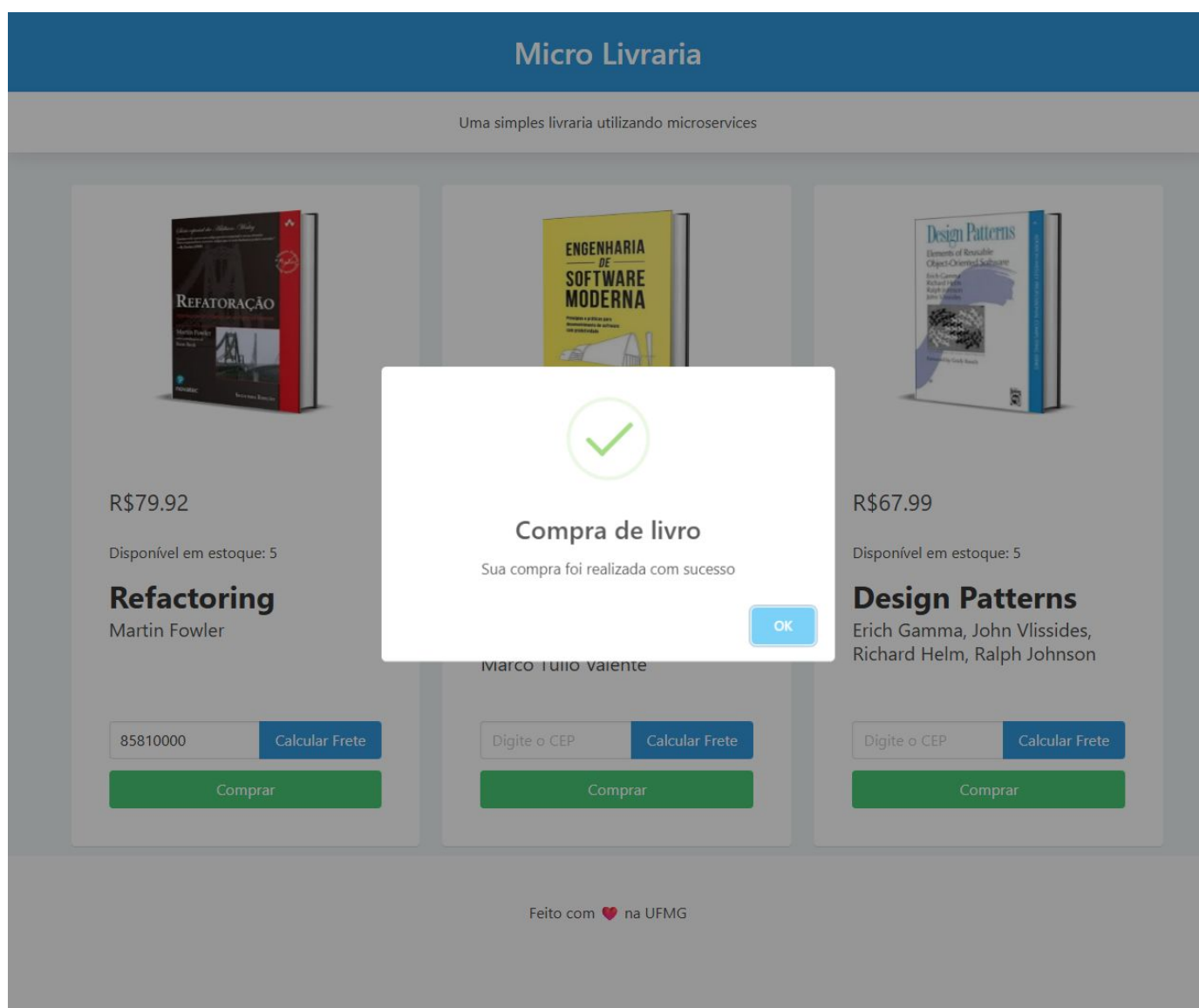
### Comando ou ação executada

Fechar a mensagem de frete > Comprar

**Explicação:** O botão Comprar mostra uma mensagem de confirmação na interface. Trata-se de uma simulação visual: não há pagamento, persistência de pedido nem atualização do estoque neste fluxo.

**Resultado e interpretação:** A interface apresentou "Sua compra foi realizada com sucesso". O resultado comprova o funcionamento do botão e da mensagem, sem representar uma compra real.

### Print do resultado



## 10. Parte 1 - testes de integração

### Comando ou ação executada

```
node --test tests/livraria.test.cjs
```

**Explicação:** Executa testes com o runner nativo do Node.js. As verificações acessam a aplicação em execução: listagem, busca dos três IDs e consulta de frete pelo Controller.

**Resultado e interpretação:** Os cinco testes passaram. Foram verificados o status HTTP, os identificadores e nomes esperados, além do formato da resposta do frete. Os testes de busca cobrem os IDs existentes no catálogo; não representam uma validação completa de entradas inválidas.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
node --test tests/livraria.test.cjs

...
# Subtest: busca o livro 3 pelo identificador
ok 4 - busca o livro 3 pelo identificador
---
  duration_ms: 6.3387
  type: 'test'
...
# Subtest: consulta o frete pelo controller
ok 5 - consulta o frete pelo controller
---
  duration_ms: 6.6479
  type: 'test'
...
1..5
# tests 5
# suites 0
# pass 5
# fail 0
```

Saída capturada do comando executado em PowerShell. Trecho 2/3.

# 11. Parte 1 - publicação

## Comando ou ação executada

```
git add .gitignore package.json package-lock.json LICENSE proto services tests/livraria.test.cjs
git commit -m "Implementa busca de livros por ID"
git push -u origin feat/aula04
```

**Explicação:** add seleciona os arquivos para o commit. commit registra a implementação no histórico. push envia a branch ao GitHub; -u associa a branch local à branch remota para os próximos envios.

**Resultado e interpretação:** O commit 4a6588e foi enviado para feat/aula04. A branch foi publicada no repositório já usado nas aulas anteriores, em vez de criar outro repositório. A licença MIT do projeto de origem foi preservada.

## Print do resultado

### Registro de execução

Micro-livraria - Aula 04

```
git push -u origin feat/aula04
```

```
remote:
remote: Create a pull request for 'feat/aula04' on GitHub by visiting:
remote:   https://github.com/lfbpaiva/DevOps/pull/new/feat/aula04
remote:
branch 'feat/aula04' set up to track 'origin/feat/aula04'.
To https://github.com/lfbpaiva/DevOps.git
* [new branch]      feat/aula04 -> feat/aula04
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

## 12. Parte 2 - verificar o Docker

### Comando ou ação executada

```
docker version
docker compose version
```

**Explicação:** docker version consulta tanto a CLI quanto o motor que executa os containers. docker compose version mostra a versão da ferramenta de composição, embora a tarefa use docker build e docker run diretamente.

**Resultado e interpretação:** Cliente e servidor responderam com Docker 29.2.1. O servidor usa Linux/amd64 no Docker Desktop 4.65.0. O Compose instalado é 5.1.0. O motor precisou de recuperação de sockets temporários antes de iniciar; isso não alterou o código da livreria.

### Print do resultado

#### Registro de execução

Micro-livraria · Aula 04

```
docker version; docker compose version
```

#### Client:

```
Version:      29.2.1
API version:   1.53
Go version:    go1.25.6
Git commit:    a5c7197
Built:         Mon Feb  2 17:20:16 2026
OS/Arch:       windows/amd64
Context:       desktop-linux
```

#### Server: Docker Desktop 4.65.0 (221669)

#### Engine:

```
Version:      29.2.1
API version:   1.53 (minimum version 1.44)
Go version:    go1.25.6
Git commit:    6bc6209
Built:         Mon Feb  2 17:17:24 2026
OS/Arch:       linux/amd64
Experimental:  false
```

Saída capturada do comando executado em PowerShell. Trecho 1/2.

#### Registro de execução

Micro-livraria · Aula 04

```
docker version; docker compose version
```

#### containerd:

```
Version:      v2.2.1
GitCommit:    dea7da592f5d1d2b7755e3a161be07f43fad8f75
```

#### runc:

```
Version:      1.3.4
GitCommit:    v1.3.4-0-gd6d73eb8
```

#### docker-init:

```
Version:      0.19.0
GitCommit:    de40ad0
```

Docker Compose version v5.1.0

Saída capturada do comando executado em PowerShell. Trecho 2/2.

## 13. Parte 2 - receita da imagem

### Comando ou ação executada

```
Get-Content shipping.Dockerfile
Get-Content .dockerignore
```

**Explicação:** O Dockerfile define a base node:22-alpine, o diretório /app, a instalação com npm ci --omit=dev e o comando de início do Shipping. As instruções COPY levam apenas dependências, contrato, código do frete e licença para a imagem.

**Resultado e interpretação:** A imagem contém o necessário para executar o frete, sem incorporar o frontend, os testes ou os arquivos Python da aula anterior. O .dockerignore exclui dependências locais, Git, ambientes virtuais, logs, arquivos .env e PDFs. npm ci utiliza o lockfile e omite dependências de desenvolvimento.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
Get-Content shipping.Dockerfile; Get-Content .dockerignore; node -p "require('./package.json').scripts.start"
```

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY proto/shipping.proto ./proto/shipping.proto
COPY services/shipping/index.js ./services/shipping/index.js
COPY LICENSE ./LICENSE
CMD ["node", "/app/services/shipping/index.js"]
node_modules/
.git/
.venv/
__pycache__/
.pytest_cache/
.ruff_cache/
.coverage
.env
*.log
*.pdf
```

Saída capturada do comando executado em PowerShell. Trecho 1/2.



## 14. Parte 2 - construir a imagem

### Comando ou ação executada

```
docker build --progress=plain -t micro-livraria/shipping -f shipping.Dockerfile ./
```

**Explicação:** build constrói a imagem. -t define o nome e -f escolhe o Dockerfile. ./ é o contexto da construção. --progress=plain mantém os logs em texto, facilitando o registro da execução.

**Resultado e interpretação:** A construção terminou com a imagem micro-livraria/shipping:latest exportada. A captura mostra o trecho final do build. A instalação na imagem apontou 11 vulnerabilidades nas dependências de produção; o sucesso do build não significa ausência de falhas de segurança.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
docker build --progress=plain -t micro-livraria/shipping -f shipping.Dockerfile ./
```

#10 DONE 0.0s

#11 [7/7] COPY LICENSE ./LICENSE

#11 DONE 0.0s

#12 exporting to image

#12 exporting layers

#12 exporting layers 0.9s done

#12 exporting manifest sha256:8bee960b8aed131785c2e72791f06222cf7e844415125d8406db510b87430a09 0.0s done

#12 exporting config sha256:090f634915a1ca8ebacce196d6f59511f01a7fc8916e74d715375acfe32e2d96 0.0s done

#12 exporting attestation manifest sha256:67e2b3bab78c5fd69d50106b75c16c1f7544ea4452691d1406cf73da29986368 0.0s done

#12 exporting manifest list sha256:b41d209cdf8e08015ef55bab4ec8f8891dc3d0cd9048977fd092cc26ab8af331

#12 exporting manifest list sha256:b41d209cdf8e08015ef55bab4ec8f8891dc3d0cd9048977fd092cc26ab8af331 0.0s done

#12 naming to docker.io/micro-livraria/shipping:latest done

#12 unpacking to docker.io/micro-livraria/shipping:latest

#12 unpacking to docker.io/micro-livraria/shipping:latest 0.9s done

#12 DONE 1.9s

Saída capturada do comando executado em PowerShell. Trecho 4/4.

## 15. Parte 2 - iniciar sem frete local

### Comando ou ação executada

```
npm.cmd run start
```

**Explicação:** Antes desta execução, o script start foi alterado para remover start-shipping, e os processos da parte 1 foram encerrados. Assim, a porta 3001 fica reservada ao container.

**Resultado e interpretação:** O frontend, o Inventory e o Controller iniciaram localmente. O log não inicia o Shipping, que será fornecido pelo Docker. Essa separação evita que o frete local esconda um problema de comunicação com o container.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
npm.cmd run start
```

```
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node services/inventory/index.js`
[nodemon] 2.0.7
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node services/controller/index.js`
WARNING: Checking for updates failed (use `--debug` to see full error)
INFO: Accepting connections at http://localhost:5000
Inventory Service running at http://127.0.0.1:3002
Controller Service running on http://127.0.0.1:3000
```

Saída capturada do comando executado em PowerShell. Trecho 2/2.

# 16. Parte 2 - executar o container

## Comando ou ação executada

```
docker run -dit --name shipping -p 127.0.0.1:3001:3001 micro-livraria/shipping
docker ps --filter name=shipping
docker logs shipping
```

**Explicação:** run cria o container a partir da imagem. -d o mantém em segundo plano; -i e -t habilitam entrada e terminal. --name define o nome. -p publica a porta 3001 somente no computador local. ps mostra o estado e logs exibe a saída do serviço.

**Resultado e interpretação:** O container 2205e71c5c55 ficou em estado Up, com a porta 127.0.0.1:3001 mapeada para 3001/tcp. O log confirmou Shipping Service running. Dentro do container, o serviço escuta em 0.0.0.0 para receber conexões pelo mapeamento.

## Print do resultado

Registro de execução

Micro-livraria · Aula 04

```
docker run -dit --name shipping -p 127.0.0.1:3001:3001 micro-livraria/shipping
```

2205e71c5c5511255750e0f7b549afbdcf3ff746de43f157aa2587721b7295a

Saída capturada do comando executado em PowerShell. Trecho 1/1.

Registro de execução

Micro-livraria · Aula 04

```
docker ps --filter name=shipping; docker logs shipping
```

| CONTAINER ID | IMAGE                   | COMMAND                  | CREATED      | STATUS      | PORTS                             |
|--------------|-------------------------|--------------------------|--------------|-------------|-----------------------------------|
| 2205e71c5c55 | micro-livraria/shipping | "docker-entrypoint.s..." | 1 second ago | Up 1 second | 127.0.0.1:3001->3001/tcp shipping |

Shipping Service running at http://127.0.0.1:3001

Saída capturada do comando executado em PowerShell. Trecho 1/1.

## 17. Parte 2 - testar frete pelo Controller

### Comando ou ação executada

```
curl.exe -sS -i http://localhost:3000/shipping/85810000
```

**Explicação:** A requisição continua entrando na API HTTP local, na porta 3000. O Controller consulta o Shipping via gRPC na porta 3001, agora atendida pelo container Docker.

**Resultado e interpretação:** A API retornou HTTP 200 com o CEP informado e value 35.09730529785156. O frete funciona sem o processo local de Shipping. O valor é aleatório no projeto original e pode mudar a cada consulta.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
curl.exe -sS -i http://localhost:3000/shipping/85810000
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 44
ETag: W/"2c-fMc0e02sOT79WfnwFgJCnFThDAg"
Date: Fri, 28 Aug 2026 19:52:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

```
{"cep":"85810000","value":35.09730529785156}
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

## 18. Parte 2 - interface com Docker

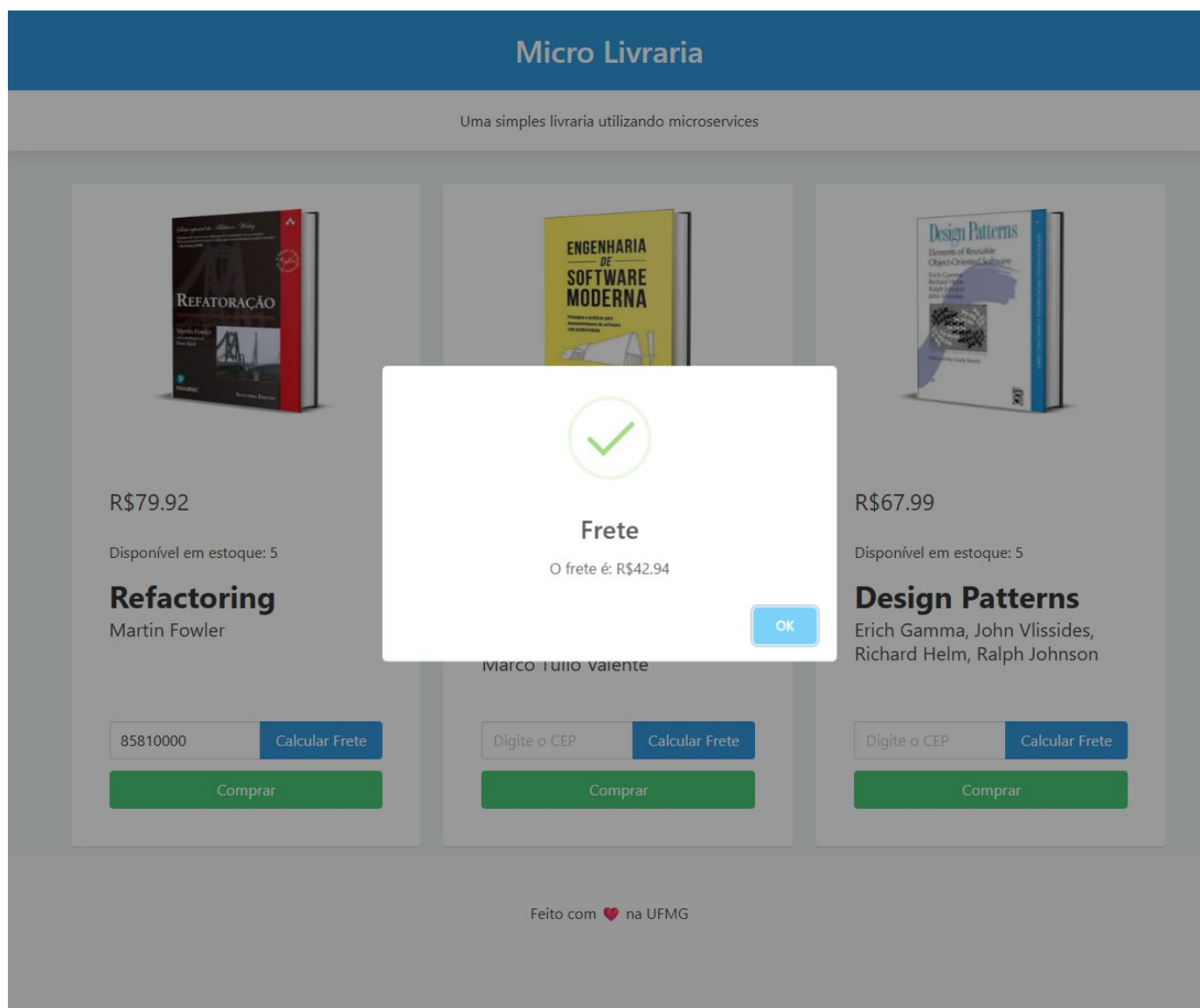
### Comando ou ação executada

```
Recarregar http://localhost:5000  
Informar CEP 85810000 > Calcular Frete
```

**Explicação:** A interface foi recarregada após a troca do serviço local pelo container. O botão usa a mesma rota HTTP; a mudança de infraestrutura não exige alteração no frontend.

**Resultado e interpretação:** A interface apresentou o frete de R\$ 42,94. A diferença em relação à consulta anterior é esperada, pois foi uma nova chamada ao serviço que gera valores aleatórios. A captura comprova o fluxo de navegador, API e frete no container.

### Print do resultado



## 19. Parte 2 - repetir os testes

### Comando ou ação executada

```
node --test tests/livraria.test.cjs
```

**Explicação:** Executa novamente a mesma suíte usada na parte 1, desta vez com o Shipping no Docker. Isso compara o comportamento externo antes e depois da mudança de ambiente.

**Resultado e interpretação:** Os cinco testes passaram novamente: listagem, buscas dos IDs 1, 2 e 3 e consulta de frete. O contrato da API foi mantido após a mudança. Os 27 testes Python da aula anterior também passaram na verificação local.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
node --test tests/livraria.test.cjs

...
# Subtest: busca o livro 3 pelo identificador
ok 4 - busca o livro 3 pelo identificador
---
  duration_ms: 4.3279
  type: 'test'
...
# Subtest: consulta o frete pelo controller
ok 5 - consulta o frete pelo controller
---
  duration_ms: 6.2809
  type: 'test'
...
1..5
# tests 5
# suites 0
# pass 5
# fail 0
```

Saída capturada do comando executado em PowerShell. Trecho 2/3.

## 20. Parte 2 - publicar a mudança

### Comando ou ação executada

```
git add .dockerignore shipping.Dockerfile package.json .github/workflows/ci.yml
git commit -m "Adiciona container para o servico de frete"
git push origin feat/aula04
```

**Explicação:** Registra a receita da imagem, os arquivos excluídos do build, a mudança no script start e o job de integração com Docker. O push atualiza a mesma branch remota da atividade.

**Resultado e interpretação:** O commit f02b3b0 foi enviado ao GitHub. O CI passou a construir a imagem, iniciar frete e serviços locais e executar a suíte de integração. Nenhum merge na main foi realizado.

### Print do resultado

#### Registro de execução

Micro-livraria · Aula 04

```
git commit -m "Adiciona container para o servico de frete"
```

```
[feat/aula04 f02b3b0] Adiciona container para o servico de frete
4 files changed, 56 insertions(+), 1 deletion(-)
create mode 100644 .dockerignore
create mode 100644 shipping.Dockerfile
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

#### Registro de execução

Micro-livraria · Aula 04

```
git push origin feat/aula04
```

```
To https://github.com/lfbpaiva/DevOps.git
4a6588e..f02b3b0 feat/aula04 -> feat/aula04
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.



## 21. Parte 2 - parar e remover o container

### Comando ou ação executada

```
docker stop shipping  
docker rm shipping
```

**Explicação:** stop encerra a execução do container. rm remove o container parado. São operações diferentes: parar não remove a instância e remover a instância não remove a imagem usada para criá-la.

**Resultado e interpretação:** Os dois comandos retornaram shipping e terminaram com sucesso. A limpeza atingiu somente o container criado nesta atividade; os containers que já existiam no computador foram preservados.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
docker stop shipping
```

```
shipping
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

#### Registro de execução

Micro-livraria - Aula 04

```
docker rm shipping
```

```
shipping
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

## 22. Parte 2 - remover a imagem e conferir

### Comando ou ação executada

```
docker rmi micro-livraria/shipping
docker ps -a --filter name=shipping
docker image ls micro-livraria/shipping
```

**Explicação:** rmi remove a imagem identificada pelo nome. As consultas seguintes verificam se ainda há um container correspondente e se a imagem permanece na listagem. A imagem base e caches de construção podem continuar armazenados pelo Docker.

**Resultado e interpretação:** A saída informou Untagged e Deleted. As listagens seguintes não mostraram o container shipping nem a imagem micro-livraria/shipping. Para executar novamente a versão final, é necessário reconstruir a imagem e criar o container.

### Print do resultado

#### Registro de execução

Micro-livraria · Aula 04

```
docker rmi micro-livraria/shipping
```

Untagged: micro-livraria/shipping:latest

Deleted: sha256:b41d209cdf8e08015ef55bab4ec8f8891dc3d0cd9048977fd092cc26ab8af331

Saída capturada do comando executado em PowerShell. Trecho 1/1.

#### Registro de execução

Micro-livraria · Aula 04

```
docker ps -a --filter name=shipping; docker image ls micro-livraria/shipping
```

```
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
IMAGE         ID          DISK USAGE  CONTENT SIZE  EXTRA
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

## 23. Verificação no GitHub Actions

### Comando ou ação executada

```
gh run view 33205879015 --repo lfbpaiva/DevOps
```

**Explicação:** Consulta o resultado da execução de CI acionada pelo envio da parte 2. A verificação ocorre no ambiente do GitHub, além dos testes realizados no computador local.

**Resultado e interpretação:** A execução terminou com sucesso: testes-pytest em 13 segundos e testes-livraria-docker em 22 segundos. Houve avisos de atualização do runtime de algumas Actions antigas, mas os dois jobs passaram. O link da execução está nas referências.

### Print do resultado

#### Registro de execução

Micro-livraria · Aula 04

```
gh run view 33205879015 --repo lfbpaiva/DevOps
```

✓ feat/aula04 CI · 33205879015

Triggered via push about 1 minute ago

#### JOBS

✓ testes-pytest in 13s (ID 98966663552)

✓ testes-livraria-docker in 22s (ID 98966663768)

#### ANNOTATIONS

! Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>  
testes-pytest: .github#2

! Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-node@v4. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>  
testes-livraria-docker: .github#4

For more information about a job, try: `gh run view --job=<job-id>`

View this run on GitHub: <https://github.com/lfbpaiva/DevOps/actions/runs/33205879015>

Saída capturada do comando executado em PowerShell. Trecho 1/1.

## 24. Histórico da atividade

### Comando ou ação executada

```
git log -3 --oneline
```

**Explicação:** Mostra os três commits mais recentes em formato resumido. Cada identificador permite consultar o estado do projeto naquela etapa.

**Resultado e interpretação:** O histórico mantém a Aula 03 no commit 994ceda, a parte 1 da Aula 04 em 4a6588e e a parte 2 em f02b3b0. Essa separação permite acompanhar a mudança do frete local para o frete executado no Docker.

### Print do resultado

#### Registro de execução

Micro-livraria - Aula 04

```
git log -3 --oneline
```

```
f02b3b0 Adiciona container para o servico de frete  
4a6588e Implementa busca de livros por ID  
994ceda Adiciona pedidos e verificacoes de qualidade da aula 03
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

# Conclusões e referências

## Conclusão

A nova rota retornou o livro solicitado por ID. O serviço de frete foi empacotado em imagem e executado no Docker, mantendo a API e o frontend funcionando com o mesmo contrato. Os cinco testes de integração passaram antes e depois da mudança. O container e a imagem da atividade foram removidos ao final, conforme o roteiro.

## Adaptações de execução

Foi usada uma branch no repositório existente, em lugar de um fork separado. A imagem usa Node 22 Alpine e instala versões do lockfile sem dependências de desenvolvimento. O container foi iniciado em segundo plano e a porta foi publicada apenas em 127.0.0.1. Essas opções mantêm o objetivo das duas tarefas e estão refletidas nos comandos registrados.

## Limitações observadas

O projeto é didático: o frete é aleatório e a compra é apenas uma confirmação visual. O contrato usa float para preços. As dependências antigas geraram alertas de segurança: 30 no ambiente completo e 11 na instalação de produção da imagem. O sistema não deve ser tratado como pronto para produção sem revisão dessas dependências e validações adicionais.

## Fontes e entrega

Roteiro e código de origem (ASERG/UFMG):  
<https://github.com/aserg-ufmg/micro-livraria>

Código da Aula 04:  
<https://github.com/lfbpaiva/DevOps/tree/feat/aula04>

Commit da parte 1:  
<https://github.com/lfbpaiva/DevOps/commit/4a6588e>

Commit da parte 2:  
<https://github.com/lfbpaiva/DevOps/commit/f02b3b0>

Execução do CI verificada:  
<https://github.com/lfbpaiva/DevOps/actions/runs/33205879015>

Material de apoio: Aula04.pdf, Prof. Lucca Abbado Neres, FAG, 2026. O código original da micro-livraria foi elaborado por Rodrigo Brito, sob orientação de Marco Tulio Valente. Licenças indicadas no projeto: MIT para o código e CC-BY para o roteiro.